CHAPTER 4

Instruction Set & Formats

Instruction parts · 0/1/2/3-address machines · RISC vs CISC · endianness · expanding opcodes

QUICK TRICK

Instruction = **opcode + operand fields**. N instructions need $\lceil \log_2 N \rceil$ opcode bits. M addressable locations need $\log_2 M$ address bits. Expanding opcode: **(unused codes)** $\times 2^{\text{field bits}}$ gives the codes for the next level.

4.1 What is an instruction set?

● LEVEL 1 · BEGINNER

The **instruction set architecture (ISA)** is the list of operations the hardware understands and how they are encoded. It is the contract between software and hardware. **Analogy.** A restaurant menu: the menu (ISA) stays the same even if the kitchen (hardware) is rebuilt.

Category	Examples
Data transfer	LOAD, STORE, MOV, PUSH, POP
Arithmetic / logic	ADD, SUB, MUL, AND, OR, SHIFT, COMPARE
Control transfer	JUMP, BRANCH, CALL, RETURN
I/O and system	IN, OUT, HALT, interrupt return

Key registers: **PC** (address of the next instruction), **IR** (current instruction), **MAR/MDR** (memory address/data), **SP** (stack pointer), flags (Z, N, C, V).

4.2 Number of addresses

● LEVEL 2 · EXAM

Task: X = (A + B) × (C + D). Count instructions in each machine type.

Type	Program	Count
3-address	ADD R1,A,B · ADD R2,C,D · MUL X,R1,R2	**3**
2-address	MOV R1,A · ADD R1,B · MOV R2,C · ADD R2,D · MUL R1,R2 · MOV X,R1	**6**
1-address (accumulator)	LOAD A · ADD B · STORE T · LOAD C · ADD D · MUL T · STORE X	**7**
0-address (stack)	PUSH A · PUSH B · ADD · PUSH C · PUSH D · ADD · MUL · POP X	**8**

QUICK TRICK

More addresses → shorter programs but longer instructions. A stack machine uses the **postfix** form: (A + B) × C becomes A B + C ×.

4.3 Instruction format sizing

● LEVEL 2 · EXAM

- Instruction length = opcode + mode bits + register fields + address / immediate fields.
- **Example.** 32-bit instruction, 6-bit opcode, three register fields for 32 registers (3 × 5 = 15 bits) → 32 − 6 − 15 = **11 bits** left over.
- Addressable memory = $2^{(\text{address field bits})}$ locations. A 10-bit address field reaches 1024 words.
- **PC-relative branch:** an 8-bit signed offset reaches −128 … +127 words from the PC.

4.4 Expanding opcode

● LEVEL 3 · GATE

A fixed-length instruction can hold many 0- and 1-address instructions by reusing unused opcode patterns as **prefixes** for the next level.

Example. 16-bit instruction, each address field 4 bits. 3-address: opcode 4 bits (16 codes). Suppose 12 are used.

```
3-address : 16 codes, 12 used -> 4 left
2-address : 4 left x 2^4 = 64 codes (8-bit opcode), say 30 used -> 34 left
1-address : 34 left x 2^4 = 544 codes (12-bit opcode)  -> max 544 one-address instructions
0-address : (whatever is left) x 2^4 ...
```

4.5 RISC vs CISC, endianness

● LEVEL 2 · EXAM

	RISC	CISC
Instruction length	Fixed (e.g. 32 bit)	Variable
Addressing modes	Few	Many
Memory access	Only LOAD/STORE	Most instructions
Registers	Many (e.g. 32)	Fewer
Control unit	Hardwired, easy to pipeline	Often microprogrammed
CPI	Close to 1 (pipelined)	Higher, varies

Endianness. Store 0x12345678 at address 100. **Big-endian** (most significant byte first): 100: 12, 101: 34, 102: 56, 103: 78. **Little-endian** (least significant first): 100: 78, 101: 56, 102: 34, 103: 12.

4.6 Common mistakes

COMMON MISTAKE

- Counting address bits for a **word**-addressable memory as if it were byte-addressable.
- Forgetting that the leftover opcode codes (not the used ones) are expanded.
- Using $\lfloor \log_2 \rfloor$ instead of $\lceil \log_2 \rceil$ for opcode bits.
- Mixing up the number of instructions needed in 1-address vs 2-address code.

GATE QUICK CHECK — Chapter 4

```
opcode bits = ceil(log2 #instr)     address bits = log2 (#locations)
(A+B)*(C+D): 3-addr 3 | 2-addr 6 | 1-addr 7 | 0-addr 8
expanding opcode: next level codes = (codes left) x 2^(field bits)
big-endian: MSB at lowest address   RISC: fixed length, load/store only
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Minimum opcode bits for 60 instructions.

Q2. Address bits for a byte-addressable 1 MB memory.

Q3. Which uses fixed-length instructions, RISC or CISC?

Q4. Write A = (B + C) × D in 1-address code and count the instructions.

● LEVEL 2 · EXAM

Q5. 32-bit instruction, 6-bit opcode, three register fields for 32 registers. Bits left over?

Q6. 0x12345678 is stored at address 100. What byte is at address 100 in little-endian?

Q7. Postfix form and stack code for (A + B) × C.

Q8. 16-bit instruction with a 6-bit opcode and one 10-bit address. How many words can it address directly?

● LEVEL 3 · GATE

Q9. 16-bit instruction, 4-bit address fields. 12 three-address and 30 two-address instructions exist. Maximum one-address instructions?

Q10. A PC-relative branch uses an 8-bit signed offset (in words). Range of reachable targets?

Q11. How many different 2-address instructions can be added if all 15 of 16 three-address codes are used (4-bit fields, 16-bit instruction)?

Answers

Q	Solution
1	$\lceil \log_2 60 \rceil$ = **6**.
2	2^{20} bytes → **20 bits**.
3	**RISC**.
4	LOAD B · ADD C · MUL D · STORE A = **4**.
5	32 − 6 − 15 = **11 bits**.
6	**0x78** (least significant byte first).
7	A B + C ×: PUSH A · PUSH B · ADD · PUSH C · MUL · POP X.
8	2^{10} = **1024**.
9	16 − 12 = 4 → 4 × 16 = 64 two-address codes; 64 − 30 = 34 → 34 × 16 = **544**.
10	**−128 … +127** words from the PC.
11	16 − 15 = 1 code left → 1 × 16 = **16** two-address codes.

Next: Chapter 5 — Addressing Modes.

AMJESH singhamjesh.github.io